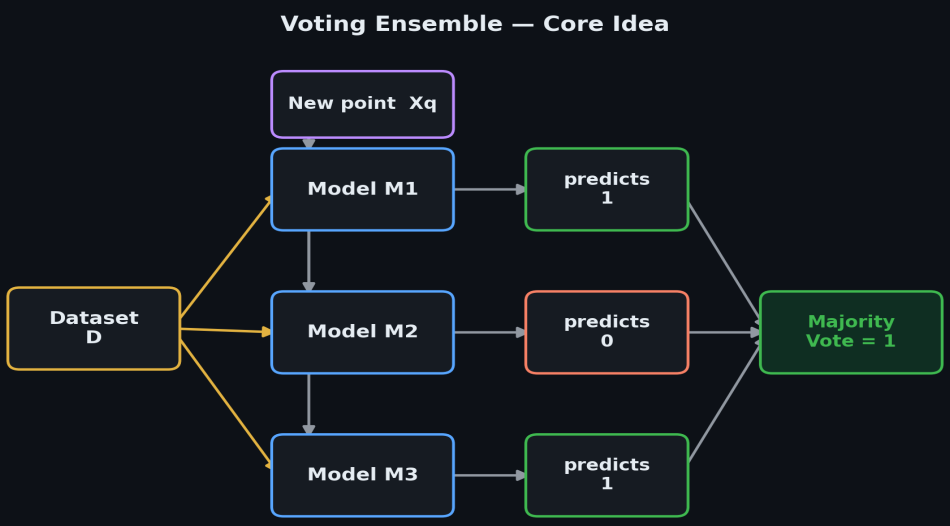
# Voting Ensembles

## From First Principles to Practical Scikit-Learn

A complete, from-scratch tutorial covering the core idea, the intuition, the mathematical proof of *why* voting works, hard vs. soft voting, weighted voting, and hands-on scikit-learn implementation.




Basic → Intermediate → Advanced

# Table of Contents


1. What Is an Ensemble?
2. The Core Idea of a Voting Ensemble
3. Why Does Voting Actually Work? (Intuition)
4. The Two Golden Assumptions
5. Mathematical Proof: The 3-Model Case
6. What Happens Below 50% Accuracy
7. Hard Voting vs. Soft Voting
8. Implementing a Voting Classifier in Scikit-Learn
9. Weighted Voting — Not All Models Are Equal
10. Two Strategies: Different Algorithms vs. Tuned Variants
11. Voting for Regression (Preview)
12. Key Takeaways & Cheat Sheet

# 1 What Is an Ensemble?

In machine learning, an **ensemble** simply means combining multiple models together so that they work as a team instead of relying on a single model. Instead of training one algorithm and hoping it captures everything about the data, you train several models and merge their outputs into one final prediction.

There are several ensemble strategies — bagging, boosting, stacking — but the simplest and most intuitive one is what we'll master in this tutorial: the **Voting Ensemble**. It's the natural place to start because the core idea requires no new machinery: you just train ordinary models the way you always do, and combine their predictions at the end.

> ◆ **Where this fits**
>
> This tutorial builds the voting ensemble from the ground up: the core idea → the intuition for why it works → a full mathematical proof → practical implementation with hard voting, soft voting, and weighted voting.

# 2 The Core Idea of a Voting Ensemble

The idea is refreshingly simple. Suppose you bring in three models — call them $\mathbf{M_1}$, $\mathbf{M_2}$, and $\mathbf{M_3}$. These can be three *different* algorithms (say, Logistic Regression, a Decision Tree, and a k-NN classifier), or they can even be the same algorithm trained with different settings. We'll see examples of both later.

All three models are trained **independently** on the **same dataset** D. None of them know about each other's existence — they simply learn from the data on their own.

Now, when a new data point $\mathbf{X_q}$ (the "query" point) arrives, you send it to *all three* trained models for a prediction. Suppose it's a binary classification problem, with classes 0 and 1:

→ Model $M_1$ predicts **1**

→ Model $M_2$ predicts **0**

→ Model $M_3$ predicts **1**

Since **two out of three** models say the answer is 1, the voting ensemble's final output is **1**. That's it — majority rules, exactly like a democratic vote.

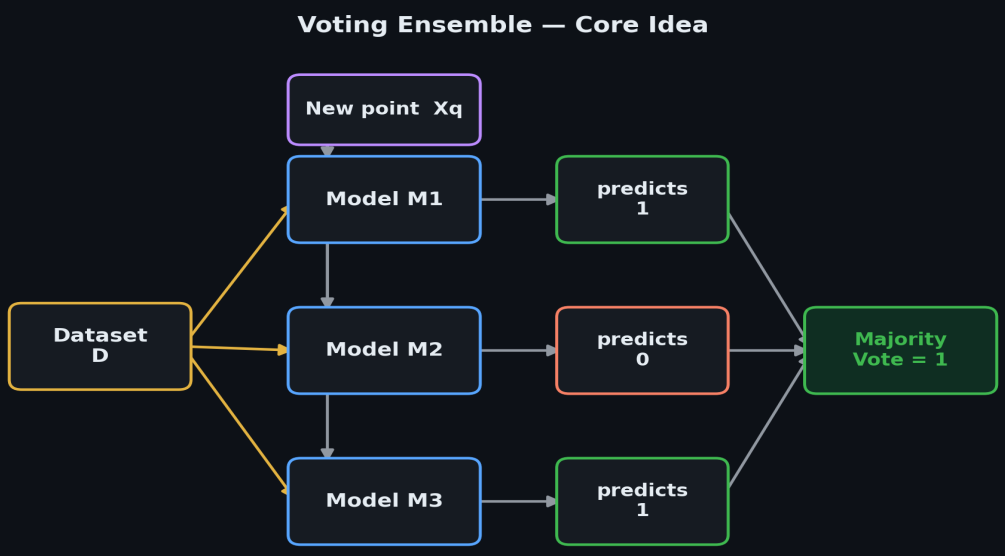



*Figure 2.1 — Three independently-trained models vote on a new point Xq; the majority class wins.*

## 2.1 What about regression?

For a regression problem, there's no "class" to vote on — the output is a continuous number. In that case, the ensemble's prediction is simply the **mean (average)** of all the individual predictions. If $M_1$, $M_2$, and $M_3$ predict 0.75, 0.92, and 1.55 respectively, the ensemble's output is:

```
(0.75 + 0.92 + 1.55) / 3  =  1.073
```

We'll cover the regression version of voting in more depth in a follow-up (see Section 11 for a preview) — the classification case is where the interesting math lives, so that's our focus here.

# 3 Why Does Voting Actually Work? (Intuition)

Here's a question worth sitting with: if one model has 70% accuracy, another has 60%, and a third has 65%, how is it even possible that *combining* them produces a model that's **more** accurate than the best individual model? Shouldn't averaging three mediocre models just give you another mediocre model?

The answer comes down to **diversity**. Think about a televised quiz show with a "phone-a-friend" style audience poll. If everyone in that audience studied Computer Science, you'll get great help on CS questions — but you're stuck the moment a history or literature question comes up. Now imagine an audience made of a CA (accountant), an engineer, a painter, and a poet — people from completely different backgrounds. If one of them doesn't know an answer, there's a good chance someone else in that diverse group does.

That's exactly the reasoning behind a voting ensemble. Every individual model has its own blind spots — patterns in the data it doesn't capture well. When the base models are **different from each other**, one model's weakness is often covered by another's strength. The mistakes don't line up, so when you combine them, the errors tend to cancel out while the correct, agreed-upon signal reinforces itself.

> ◆ **Intuition in one line**
>
> A group of diverse, individually-competent models makes fewer *correlated* mistakes than a single model — so their combined vote is more reliable than any one of them alone.

# 4 The Two Golden Assumptions

Voting isn't magic — it only works if two conditions hold. Break either one, and the ensemble can actually perform *worse* than a single model.

## Assumption 1 — The base models should be independent / diverse

The models should be as different from each other as possible — different algorithms, different feature subsets, or different hyperparameters. If all your models are near-identical (e.g. three Logistic Regressions with almost the same settings), they will make the *same* mistakes on the *same* points, and voting buys you nothing.

## Assumption 2 — Each individual model must beat 50% accuracy

This is the more surprising one. Every base model's accuracy must be **greater than 50%** (better than random guessing on a binary problem). If it drops below 50%, adding more such models to the vote doesn't help — it actively **hurts**, pulling the ensemble's accuracy below even the worst individual model. We'll prove this rigorously with real numbers in Section 6.

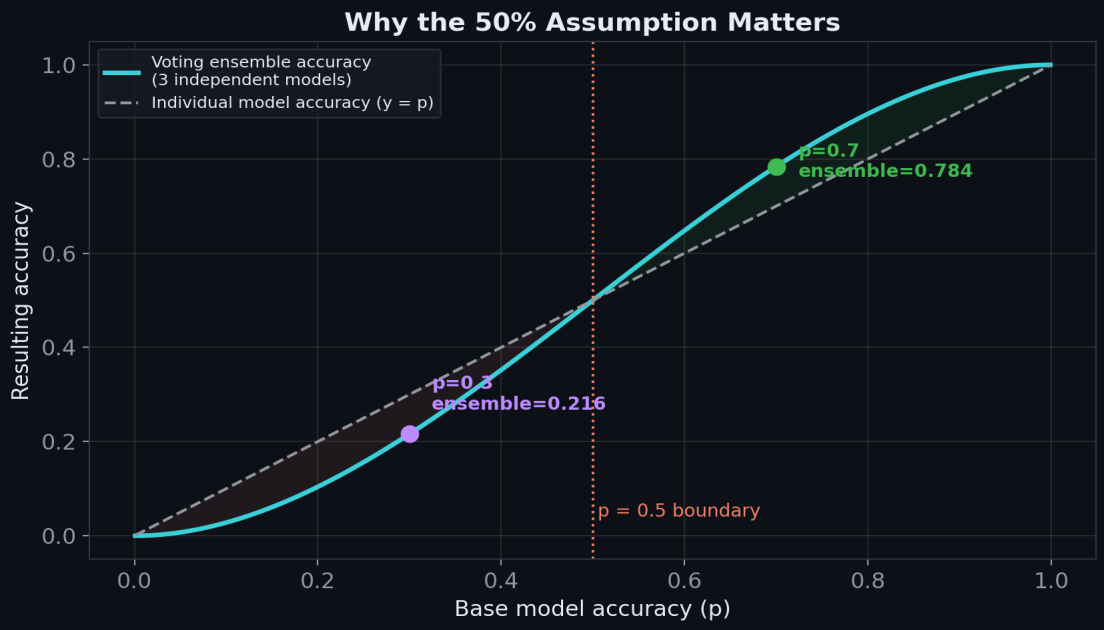



*Figure 4.1 — Ensemble accuracy vs. individual model accuracy p, for 3 independent models. Above p = 0.5 voting helps; below p = 0.5 it hurts.*

# 5 Mathematical Proof: The 3-Model Case

Let's now prove — with real probability math — that combining three "weak" models with 70% accuracy each can produce an ensemble with *higher* than 70% accuracy.

## 5.1 Setup

Assume we have three independent estimators $M_1$, $M_2$, $M_3$, each with individual accuracy **p = 0.7** on a new data point (so each has a 0.3 probability of being wrong). "Independent" means one model's correctness doesn't influence another's — their errors are uncorrelated.

For a voting classifier to produce the *correct* final answer, we need **at least 2 of the 3 models** to be correct (a majority). So we need to compute:

```
P(voting ensemble is correct) = P(at least 2 of 3 models are correct)
```

## 5.2 Enumerating every outcome

Each of the 3 models is independently either Correct (C) or Wrong (W), so there are 2 × 2 × 2 = **8 possible outcomes**. Since P(correct) = 0.7 and P(wrong) = 0.3 for every model, we multiply along each branch:

| M1 | M2 | M3 | Probability | ≥2 correct? |
|---|---|---|---|---|
| C | C | C | 0.7 × 0.7 × 0.7 = 0.343 | ✓ Yes |
| C | C | W | 0.7 × 0.7 × 0.3 = 0.147 | ✓ Yes |
| C | W | C | 0.7 × 0.3 × 0.7 = 0.147 | ✓ Yes |
| W | C | C | 0.3 × 0.7 × 0.7 = 0.147 | ✓ Yes |
| C | W | W | 0.7 × 0.3 × 0.3 = 0.063 | ✗ No |
| W | C | W | 0.3 × 0.7 × 0.3 = 0.063 | ✗ No |
| W | W | C | 0.3 × 0.3 × 0.7 = 0.063 | ✗ No |
| W | W | W | 0.3 × 0.3 × 0.3 = 0.027 | ✗ No |

As a sanity check, all 8 probabilities add up to exactly 1.0 (0.343 + 0.147×3 + 0.063×3 + 0.027 = 1.0), confirming we've covered every possible outcome.

## 5.3 Summing the winning cases

We now add up the probabilities of every outcome where **at least 2 models are correct** — the four highlighted rows above:

```
P(ensemble correct) = 0.343 + 0.147 + 0.147 + 0.147
                    = 0.784
```

**◆ Result**

Three individual models with 70% accuracy each combine into a voting ensemble with **78.4% accuracy** — noticeably higher than any single model! This is the mathematical proof that voting works.

## 5.4 The general formula

What we just computed by hand is a special case of the **Binomial distribution**. If each of 3 independent models has accuracy p, the ensemble's accuracy is:

$$P(\text{ensemble correct}) = 3p^2(1-p) + p^3 = 3p^2 - 2p^3$$

Plugging in p = 0.7: 3(0.49)(0.3) + 0.343 = 0.441 + 0.343 = **0.784** — matching what we computed the long way by hand. Figure 4.1 (above) plots this exact formula across every value of p from 0 to 1, and you can see the ensemble curve pulls above the y = p line whenever p > 0.5.

## 6 What Happens Below 50% Accuracy

Now let's test the second assumption. What if each of our 3 models has accuracy **p = 0.3** — worse than a coin flip? We redo the exact same calculation, just swapping which outcomes count as "correct":

| M1 | M2 | M3 | Probability | ≥2 correct? |
|---|---|---|---|---|
| C | C | C | 0.3 × 0.3 × 0.3 = 0.027 | ✓ Yes |
| C | C | W | 0.3 × 0.3 × 0.7 = 0.063 | ✓ Yes |
| C | W | C | 0.3 × 0.7 × 0.3 = 0.063 | ✓ Yes |
| W | C | C | 0.7 × 0.3 × 0.3 = 0.063 | ✓ Yes |
| C | W | W | 0.3 × 0.7 × 0.7 = 0.147 | ✗ No |
| W | C | W | 0.7 × 0.3 × 0.7 = 0.147 | ✗ No |
| W | W | C | 0.7 × 0.7 × 0.3 = 0.147 | ✗ No |
| W | W | W | 0.7 × 0.7 × 0.7 = 0.343 | ✗ No |

```
P(ensemble correct) = 0.027 + 0.063 + 0.063 + 0.063
                    = 0.216
```

**◆ Result — the flip side**

Three models with 30% accuracy each combine into a voting ensemble with only **21.6% accuracy** — worse than any of the individual models! Below the 50% line, voting amplifies the mistakes instead of cancelling them out.

This confirms Assumption 2 from Section 4: your base models must individually do better than random guessing (accuracy > 50% for a binary problem), or the voting mechanism actively works against you. The good news: beating 50% on a binary task is a fairly low bar, so this assumption is easy to satisfy in practice.

# 7 Hard Voting vs. Soft Voting

So far we've only discussed one style of voting — counting class labels. This is called **hard voting**. But there's a second, often more powerful style called **soft voting**. Let's compare them directly.

## 7.1 Hard voting

In hard voting, every model casts a vote for a discrete class label, and the class with the most votes wins — exactly the mechanism we used in Sections 2, 5 and 6.

→ $M_1$ votes **class 1**

→ $M_2$ votes **class 0**

→ $M_3$ votes **class 1**

→ Majority → final prediction is **class 1**

## 7.2 Soft voting

In soft voting, instead of a hard class label, each model outputs a **probability** for every class. The ensemble then **averages** these probabilities across all models, and whichever class has the highest average probability wins.

For example, suppose our 3 models output the following probabilities for a binary problem:

| Model | P(class 0) | P(class 1) |
|---|---|---|
| M1 | 0.20 | 0.80 |
| M2 | 0.60 | 0.40 |
| M3 | 0.14 | 0.86 |
| Average | 0.31 | 0.69 |

Since the average P(class 1) = 0.69 is higher than average P(class 0) = 0.31, soft voting predicts **class 1** — and notice it reaches the same final answer as hard voting here, but it does so by weighing *how confident* each model is, not just its final label.

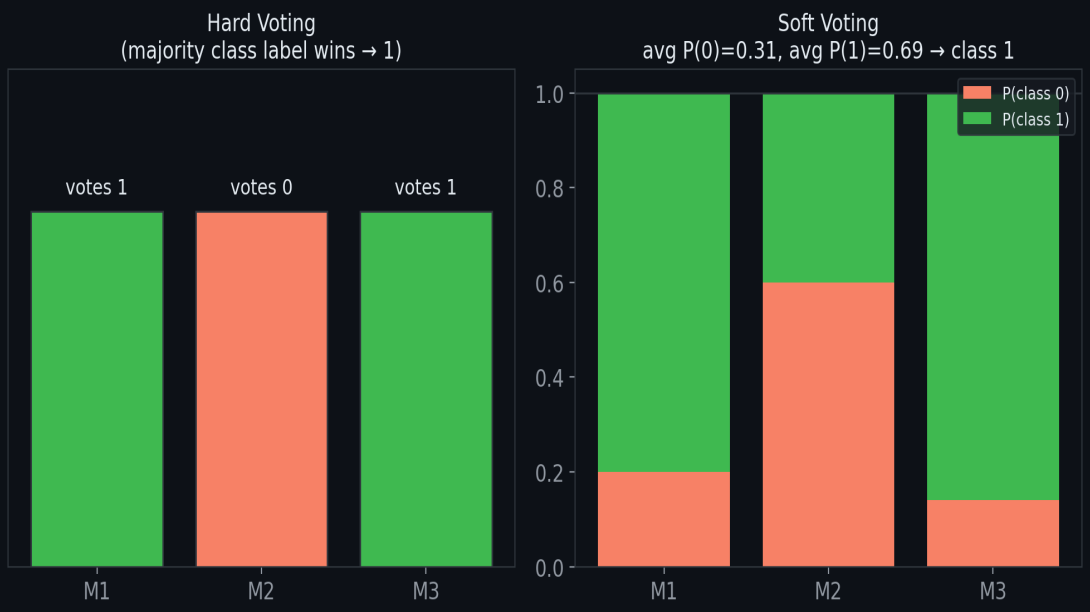



*Figure 7.1 — Hard voting counts labels; soft voting averages class probabilities.*

## 7.3 Multi-class example

The same idea extends cleanly to multi-class problems. Suppose there are 3 classes and 5 models. Soft voting simply averages the probability each model assigns to every class, then picks the class with the highest average. If class 1 ends up with an average probability of 0.55 while classes 0 and 2 get 0.30 and 0.15 respectively, class 1 wins.

## 7.4 Which one is better?

**Generally, soft voting outperforms hard voting** — because it uses more information (confidence levels, not just final labels), it tends to produce smoother, better generalized decisions.

> **◆ But not always!**
>
> Soft voting isn't automatically superior. Consider this real scenario: Logistic Regression scores 60%, Naive Bayes scores 60%, and Random Forest scores 90%. If Logistic Regression and Naive Bayes are both very *confident* but *wrong* on a point, their strong (and incorrect) probability scores can outweigh Random Forest's correct but less confident prediction — dragging soft voting's accuracy down. In that situation, hard voting (simple majority) can actually beat soft voting. **The practical rule: try both, and pick whichever performs better via cross-validation on your own dataset.**

# 8 Implementing a Voting Classifier in Scikit-Learn

Scikit-learn makes voting ensembles almost trivial with the `VotingClassifier` class. Let's build one step by step.

## 8.1 Basic setup

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

# Define the base models as (name, estimator) tuples
estimators = [
    ('lr',  LogisticRegression()),
    ('svc', SVC(probability=True)),   # needed for soft voting
    ('rf',  RandomForestClassifier())
]

# Hard voting
voting_clf = VotingClassifier(estimators=estimators, voting='hard')
scores = cross_val_score(voting_clf, X, y, cv=10)
print("Hard voting accuracy:", scores.mean())

# Soft voting
voting_clf_soft = VotingClassifier(estimators=estimators, voting='soft')
scores_soft = cross_val_score(voting_clf_soft, X, y, cv=10)
print("Soft voting accuracy:", scores_soft.mean())
```

**◆ Important detail**

For `voting='soft'` to work, every base estimator must be able to output class probabilities via `predict_proba()`. That's why SVC needs the explicit `probability=True` flag — it's off by default.

## 8.2 A concrete worked example

A great way to see voting in action is to deliberately pick a *hard* dataset. Take the classic Iris dataset, but instead of using all 3 species and all 4 features, keep only 2 species (versicolor and virginica — the two that overlap the most) and just 2 numeric features. This creates a task where no single simple model draws a clean boundary.

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame

# Keep only classes 1 and 2 (versicolor, virginica) -- the overlapping pair
df = df[df['target'] != 0]

# Keep just two (harder, overlapping) numeric feature columns
X = df[['petal length (cm)', 'petal width (cm)']]
y = df['target']
```

Now compare a single Logistic Regression against a voting ensemble of Logistic Regression + Random Forest + SVM on this harder, overlapping dataset:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

estimators = [
    ('lr', LogisticRegression()),
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True))
]

single_lr = LogisticRegression()
voting = VotingClassifier(estimators=estimators, voting='soft')

print("Logistic Regression alone:", cross_val_score(single_lr, X, y, cv=10).mean())
print("Voting ensemble:          ", cross_val_score(voting, X, y, cv=10).mean())

# Typical result:
# Logistic Regression alone: ~0.60
# Voting ensemble:           ~0.63 to 0.90+ (varies by base models)
```

As you add more diverse models (Logistic Regression → + SVM → + Random Forest), the decision boundary stops being a single straight line and becomes a flexible, combined boundary that can bend around the overlapping region — this is exactly the diversity effect from Section 3, visualized:

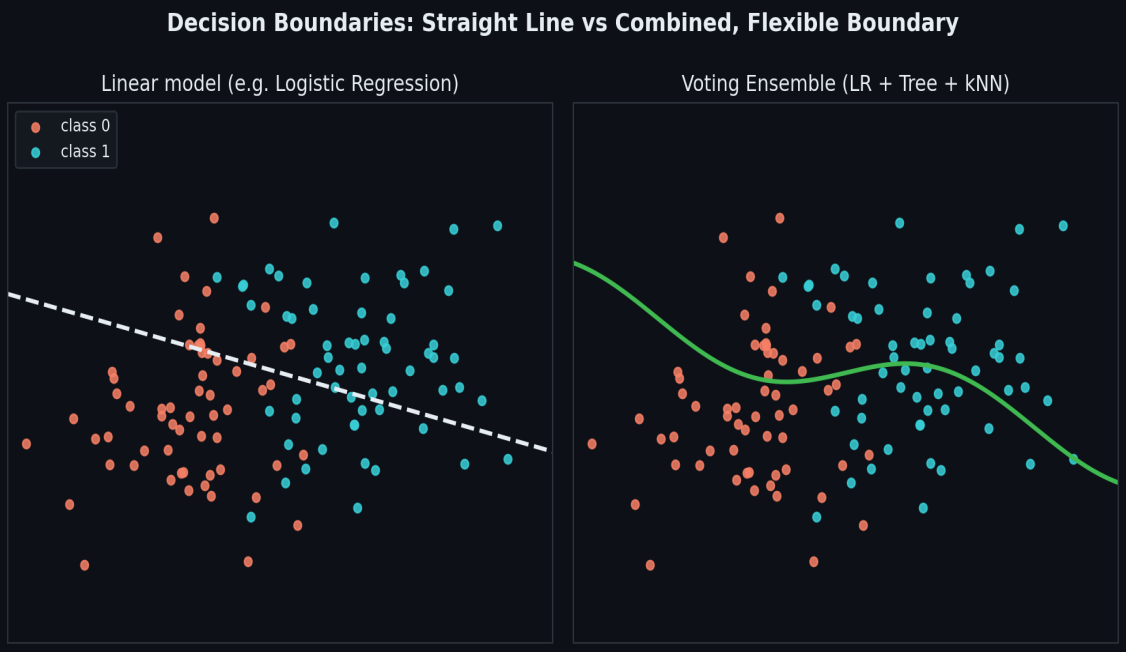



*Figure 8.1 — A single linear model can only draw a straight boundary; combining diverse models lets the ensemble adapt to the true shape of the data.*

# 9 Weighted Voting — Not All Models Are Equal

By default, every base model gets an **equal say** in the vote. But what if one of your models is clearly stronger than the others? Scikit-learn lets you assign a `weights` parameter, giving some models more influence over the final decision.

```
voting = VotingClassifier(
    estimators=estimators,
    voting='soft',
    weights=[3, 1, 1]   # M1 (e.g. the strongest model) counts 3x as much
)
```

Internally, for soft voting, this means the weighted *average* of probabilities is computed instead of a plain average — model 1's probability contributes 3 times as much as each of the others. For hard voting, model 1's vote is effectively counted 3 times when tallying the majority.

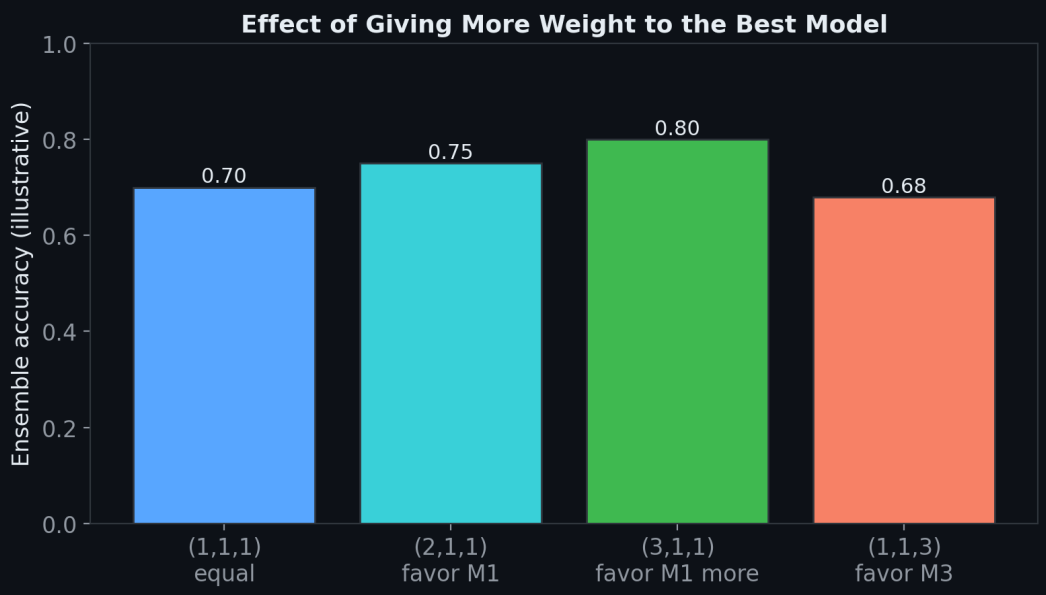



*Figure 9.1 — Illustrative effect of shifting weight toward the strongest base model. Push too far, or weight the wrong model, and accuracy can drop again.*

## 9.1 Searching for the best weights

In practice, you rarely know the best weight combination in advance — so you search for it, just like any other hyperparameter:

```
best_score, best_weights = 0, None

for w1 in range(1, 4):
    for w2 in range(1, 4):
        for w3 in range(1, 4):
            clf = VotingClassifier(estimators=estimators, voting='soft',
                                   weights=[w1, w2, w3])
            score = cross_val_score(clf, X, y, cv=10).mean()
            if score > best_score:
                best_score, best_weights = score, (w1, w2, w3)

print("Best weights:", best_weights, "-> accuracy:", best_score)
```

**◆ Rule of thumb**

Give more weight to whichever base model performs best on its own (measured with cross-validation) — but always verify with a search rather than guessing, since the interaction between models is not always intuitive.

# 10 Two Strategies: Different Algorithms vs. Tuned Variants

There are two complementary ways to build the base models for your voting ensemble:

## Strategy A — Combine different algorithms

Use fundamentally different algorithm families (Logistic Regression, Random Forest, SVM, k-NN, Naive Bayes...) as the base models. This maximizes diversity, since different algorithms tend to make different kinds of mistakes — which is exactly what voting needs (Section 3).

## Strategy B — Combine tuned variants of the same algorithm

Alternatively, take a *single* algorithm and train several versions of it with different hyperparameters — for example, an SVM with a polynomial kernel of degree 1, another of degree 3, another with an RBF kernel, and so on. Instead of picking just one "best" hyperparameter setting via trial and error, combine several good candidates into a voting ensemble:

```
from sklearn.svm import SVC

svc_variants = [
    ('svc_deg1', SVC(kernel='poly', degree=1, probability=True)),
    ('svc_deg3', SVC(kernel='poly', degree=3, probability=True)),
    ('svc_deg5', SVC(kernel='poly', degree=5, probability=True)),
    ('svc_rbf',  SVC(kernel='rbf', probability=True)),
]

voting_svc = VotingClassifier(estimators=svc_variants, voting='soft')
print(cross_val_score(voting_svc, X, y, cv=10).mean())
# Often outperforms any single hand-picked hyperparameter setting
```

**◆ When to use which**

Use **Strategy A** as your default — different algorithms give the most diversity for the least effort. Reach for **Strategy B** when you're already deep into tuning one algorithm's hyperparameters and can't settle on a single best setting — instead of picking one, let several good candidates vote together.

## 11 Voting for Regression (Preview)

Everything in this tutorial focused on classification because that's where the probability math is richest — but the same core idea (Section 2.1) applies directly to regression using scikit-learn's `VotingRegressor`:

```
from sklearn.ensemble import VotingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor

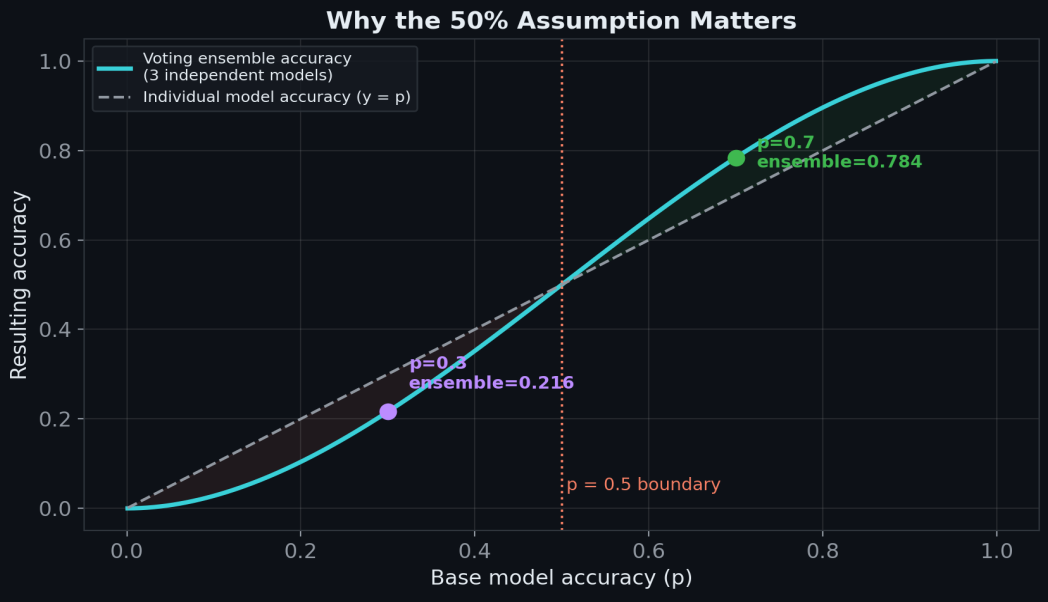
reg_estimators = [
    ('lin', LinearRegression()),
    ('tree', DecisionTreeRegressor()),
    ('knn', KNeighborsRegressor())
]

voting_reg = VotingRegressor(estimators=reg_estimators)
voting_reg.fit(X_train, y_train)
preds = voting_reg.predict(X_test)   # = average of all 3 models' predictions
```

There's no "hard" vs "soft" distinction for regression — the ensemble output is always the (optionally weighted) mean of the base models' numeric predictions, exactly as described in Section 2.1.

# 12 Key Takeaways & Cheat Sheet

- → **Core idea:** train several models independently on the same data, then combine their predictions — majority vote for classification, average for regression.
- → **Why it works:** diverse models make different mistakes; combining them cancels out uncorrelated errors while reinforcing agreed-upon signal.
- → **Assumption 1:** base models must be diverse / independent, not near-duplicates of each other.
- → **Assumption 2:** each base model must individually beat 50% accuracy (binary case) — below that, voting actively hurts.
- → **Hard voting:** majority of class labels wins. **Soft voting:** average predicted probabilities, then pick the highest — usually better, but not always.
- → **Weighted voting:** give stronger models more influence with the `weights` parameter; search for the best combination.
- → **Two build strategies:** combine different algorithms (max diversity) OR combine several tuned variants of the same algorithm.
- → Always validate hard vs. soft, and any weighting scheme, with cross-validation on your own dataset — theory tells you what's *possible*, not what's guaranteed.




*The one chart to remember: above 50% individual accuracy, voting lifts you up; below it, voting drags you down.*

*End of tutorial. Next up: a dedicated deep-dive into Voting Regressors, worked through with a full regression example.*